01 / ASSESSMENT OVERVIEW

Security Self-Assessment

2026 FinTechathon International Track
Topic E - SME Finance Copilot

Assessment method: implementation review and reproducible tests in an isolated environment. Independent penetration testing and regulatory certification are outside the scope of this assessment.

Scope and protected assets

The assessed boundary is the finance copilot: authenticated APIs, tenant/shop separation, owner approvals, model-choice validation, cost controls, audit evidence and simulator isolation. Protected assets are sales/debt/expense records, stock, cash balances, policy versions, application schedules, tokens, provider credentials and business-sensitive exports.

Trust zone	Security assumption
Browser and user input	Requests, tenant headers, shop identifiers, free text and model-visible business data are untrusted. UI-disabled controls do not authorize an operation.
Backend and database	Trusted application code enforces roles, ownership, scoped references and transactional invariants. Database administrators retain privileged access to audit records.
Model and external providers	Provider output is untrusted input. No model may authorize a financial write, bypass an approval, or replace deterministic money calculations.
Sandbox / evidence tooling	Only synthetic or explicitly selected demo records should be used. Local evidence runs must not receive a production connection string.

Verification summary

Database integration testing passed 11 tests across five suites. Separate live-provider checks exercised the AI runtime and paired-replay workflow. The accompanying evidence reports record test outcomes and the source revision used.

The assessment covers the finance copilot. Production firewall and TLS configuration, backup retention and restoration, and a full application-wide security audit have not been verified. These limitations are included in the risk register.

02 / PERMISSIONS

Access Control

Role permissions, tenant membership and shop-level authorization for finance endpoints.

Capability	Unauth.	Employee	Owner	Admin
Read finance metrics, offers, policies, runs, audit/evaluation exports	Denied	Assigned shop	Owned tenant	Trusted role
Acknowledge or dismiss a cash alert	Denied	Assigned shop	Owned tenant	Trusted role
Run matching; negotiate; accept/reject offers	Denied	Denied	Allowed	Allowed
Create cash records; backfill; approve treasury plans; link payment evidence	Denied	Denied	Allowed	Allowed
Generate/verify insights; approve/measure capital decisions; edit mandates	Denied	Denied	Allowed	Allowed
Run a matched evaluation; prepare/advance/reset demo workspace	Denied	Denied	Allowed	Allowed

Authorization sequence

1. Authentication and role: auth verifies a JWT and checks the route's explicit allowed roles. **2. Tenant membership:** verifyTenantAccess treats x-tenant-id as a requested scope, not proof; owners must own that tenant and employees must belong to it. **3. Shop assignment:** employee finance reads require an active assignment for the supplied shop. **4. Object scope:** service lookups bind referenced records to tenant and shop before mutation or export.

ADMIN is deliberately trusted across tenant membership checks; that is an administrative trust boundary, not ordinary tenant isolation. SUPER_ADMIN is recognized by tenant middleware but is not automatically admitted by finance routes, which explicitly list ADMIN, OWNER and/or EMPLOYEE. Credentials for privileged roles require separate operational protection.

Approval is not execution. Owner approval records a plan or application. A recorded collection must link an existing valid payment; accepting a financing application does not set a disbursement timestamp. No real bank movement is performed.

Audit downloads contain business-sensitive scoped data and are also available to authorized employee readers. Restrict demo access and do not treat a downloaded JSON file as public or anonymized merely because its filename says evidence.

03 / PREVENTIVE CONTROLS

AI and Data Security Controls

Input validation, financial safeguards, information handling and cost controls.

Threat / failure	Implemented boundary	Residual limitation
Prompt injection or invented options	Zod request/output validation; feasible option sets; IDs checked against the frozen universe; financial guards run independently of model instructions.	Semantic errors can remain within valid options. Guarded output is not necessarily useful or optimal.
Unauthorized financial change	Owner/admin mutation routes; scoped references; approval state transitions; unique payment evidence and transactional application creation.	A compromised owner/admin remains powerful; no claim of multi-party approval or bank authorization.
Duplicate spend or competing requests	Per-tenant monthly reservations use PostgreSQL advisory locking; evaluations claim a unique frozen input record before calling AI.	Crash-held reservations may need reconciliation; configured estimates do not replace provider billing controls.
Data leakage to a provider	Shared redaction removes selected sensitive field patterns; finance prompts favor aggregates and opaque identifiers; keys remain server-side.	Free text is not guaranteed anonymous. Third-party handling/retention needs project-owner review.
Fabricated financial results	PMT, projection, metrics, costs and replay outcomes are computed in code; null outcomes stay pending; negative deltas retained.	Models and heuristics are simplified. Reproducibility does not establish causality or real-world profit.
Test overwrite of production	Evidence URL restricted to 127.0.0.1/simulator_test with no host overrides; fresh schema; paid AI disabled in integration suites.	Local schema permissions and storage cleanup are operator responsibilities.

Operational resilience of the AI path

The runtime has deadlines, a maximum of one repair retry and deterministic degradation for unavailable credentials, schema failures, budget denial or rate limiting. Pillar feature flags can disable external AI. Usage/status and some decision records are best-effort writes; logging failure does not block the feature. Error output avoids raw provider responses and secrets.

Mandate versions and recorded choices support traceability. They are application-enforced history, not a cryptographically immutable or independently witnessed audit ledger.

04 / VERIFICATION

Security Verification

Observed results from local PostgreSQL and loopback HTTP tests using synthetic accounts.

Recorded suite	Tests	Security/correctness evidence
finance-completion	7 passed	Cash trigger write/edit/delete and rollback; concurrent cash writes; repeat backfill; expense alert; supplier obligation/payment linkage.
finance-hardening	1 passed	Employee mutation denial and cross-shop denial; foreign scope rejection; immutable mandate versions; concurrent budget cap; unique collection evidence; idempotent application; non-causal outcome labels.
inventory-finance	1 passed	Actual inventory/finance services and routes validate product, purchase and alert behavior against a migrated database.
simulator-workspace	1 passed	Dedicated demo-only fast-forward/reset; source history unchanged; concurrent lock/rollback; virtual timestamps; copied history and reset isolation.
matched-evaluation	1 passed	Frozen data, pre-outcome choices, scoped export, concurrent paid-call deduplication, unknown input rejection, JSONB round-trip replay and unchanged source financial records.

Additional verification

Live-provider benchmark: three cases completed with no fallback, with model/token/cost metadata. Shop-specific live check: a complete eight-path sandbox result using gpt-4o-mini, with no production records. Offline replay: canonical outcome hash matched the exported result. A hash match establishes internal reproducibility, not original authorship or protection from a privileged database rewrite.

Verification scope and limitations

No independent penetration test, full application-wide endpoint audit, load/DoS assessment, dependency vulnerability certification, privacy-impact assessment, firewall verification, restore drill or measured recovery objective is claimed. The execution log is not a screenshot collection or demo video. Real-world financial alpha is not demonstrated.

Detailed results are included in the submitted execution reports. All five integration suites passed without skipped tests; migration application also passed.

05 / RISK REGISTER

Known Risks and Mitigations

Qualitative priorities based on implementation review and current verification coverage.

ID / priority	Known risk / basis	Mitigation and next action
R1 / High	Browser access tokens live in localStorage. A successful same-origin script injection could expose them. No exploit was demonstrated.	Protect against XSS and review CSP/session architecture before sensitive production use; consider an HttpOnly session design. Existing role checks do not mitigate stolen valid tokens.
R2 / High	Business data and free-text prompts may contain PII; regex/field redaction is incomplete. Exports can reveal financial detail.	Use synthetic demonstration data. Review provider retention/consent, minimize fields, restrict downloads and define retention/deletion controls before real customer onboarding.
R3 / High	Trusted administrators bypass tenant membership checks. Operators can alter audit history; best-effort writes may leave gaps.	Restrict privileged access; monitor audit-write failures. Independent log retention/signing and stronger separation of duties are not established.
R4 / Medium	No global request rate limiter or comprehensive security-header middleware is configured in the inspected Express bootstrap. Edge protection was not verified.	Confirm reverse-proxy/edge limits and headers; test authentication and expensive-operation abuse paths. AI budget control alone is not DoS protection.
R5 / High	Restore capability and backup retention were not verified. Code rollback does not undo schema changes.	Run a protected staging restore drill; define off-host encrypted backups and retention; set and measure RPO/RTO. Review migration compatibility before release.
R6 / Medium	Budget reservations may remain after a crash; recorded API prices are estimates and may drift.	Reconcile stale reservations carefully; monitor provider billing and configure provider-side caps. Never silently erase charge history.
R7 / Medium	Heuristics, Float money, approximate APR and simplified counterfactuals can mislead. Offer acceptance does not re-underwrite current affordability.	Revalidate final terms and current affordability before real lending. Retain rounding checks; do not claim regulated APR, bank reconciliation or causal gains.
R8 / Medium	Legacy modules, dependencies, actual edge configuration and license inventory are outside the demonstrated finance test coverage.	Perform a whole-app security/dependency/license review before wider use; grant judges access to private source and avoid redistributing unlicensed data.

06 / OPERATIONS AND RECOVERY

Operational Resilience

Deployment safeguards, recovery procedures and operational limitations.

Implemented deployment safeguards

The backend workflow builds and tests an artifact, uses a restricted SSH deployment identity and verifies a release checksum. The deployment script installs dependencies, generates Prisma, starts bizwizard-backup.service, applies committed migrations and tests a database connection before switching the current release. It checks HTTP readiness after restart and attempts to restore the previous code release if readiness fails.

Unverified recovery controls: backup policy, backup integrity, off-host copies and restoration require operational validation. RPO and RTO have not been measured. The readiness endpoint checks availability rather than financial integrity.

Recovery procedure

Step	Required operator action
Contain	Disable affected writes/external AI, retain incident logs, identify the affected release/tenant and rotate compromised credentials. Do not publish secret-bearing errors.
Select recovery point	Choose a verified backup and matching code/migration version. Preserve the incident database; do not overwrite production to experiment.
Restore in isolation	Restore PostgreSQL into an isolated staging environment. Verify schema, cash source links, balances, schedules, tenant boundaries and policy/evaluation records. Measure elapsed time and data gap.
Recover and validate	Promote only after operator approval. Recheck readiness and authenticated user flows; reconcile financial records; record actual RPO/RTO and any data loss. Coordinate schema rollback separately from code rollback.

Data handling and compliance

Use only authorized synthetic/demo data in public evidence. Keep environment files and credentials out of Git; provide private repository access explicitly to judges. Review third-party package/data rights and notices. No root project license was found in the inspected tracked files; private review access is not an open-source license. Financial, privacy and credit-law compliance requires a separate assessment.

References and supporting material

Assessment requirement: FinTechathon International Track competition appendix, submission checklist (page 3).
Implementation: project repository and root README; finance authorization, validation, budget and deployment controls.
Verification: the reports in evidence/submission/ record integration results, live-provider checks and replay verification, together with the tested revision. The technical documentation describes the system architecture and algorithms.